Project 3: Evolutionary Algorithms

T504 Machine Learning / T711 Fundamentals of Machine Learning

Fall, 2026

1 Introduction

The goal of this project is to apply your theoretical knowledge about genetic algorithms and their components. In particular, the task is to implement a simple genetic algorithm (SGA) for binary representation from scratch (i.e., code the required functions for all elements of the SGA in Python yourself) and adjust it for two different types of tasks. You need to also hand in a report outlining the steps, describing and explaining the functions, justifying all design decisions, and presenting the results.

This project can be done in groups of up to 3 students (2 recommended, alone possible). (The groups do not have to be the same as for the labs.)

Note: Please make sure that you briefly check each function after you have defined it with simple cases to make sure that they work as intended (In some cases, this is specifically requested as a subtask but this is in general a practice to follow). Apart from helping you verify that functions work correctly, it is also required to perform these checks and print the results in the Jupyter Notebook to help with the grading. Thank you!

2 Data Sets

Task 1 does not require a dedicated data set. Knowledge of the input space (i.e., what values can be inputs) and the fitness function is enough.

Task 2 & 3 use the *"data_1001"* data set, representing a knapsack problem with 1001 items, in which each item has a given "Value" and "Weight".

Note: Please set and report the **random seed** once near the beginning of the jupyter notebook. Use the last five digits of the smallest student number in the group as the seed. Do not reset it during an SGA run. This is done for reproducibility so we can run your (stochastic) algorithm and get the same result.

```
import random
import numpy as np

seed = 12345  # Replace with seed number as stated above
random.seed(seed)
np.random.seed(seed)
```

3 Task 1: 6-bit Representation of Integers

This first task is about programming a simple genetic algorithm (SGA) from scratch - coding individual functions (in Python) to perform key functionalities for the SGA, and then using these functions in a loop to create the SGA. This task assumes that integer values are encoded as 6-bit binary strings (represented for convenience as a list) e.g., $[0, 0, 0, 0, 1, 1]$ is 3, encoding values as integers from 0 to 63.

For going from the phenotype to the genotype with 6-bit strings as well as back you can use the following functions (copy them into your Jupyter notebook):

```
def geno_to_pheno(individuals):
    return [int("".join(map(str, individual)), 2) for individual in individuals]

def pheno_to_geno(x_values):
    return [[int(bit) for bit in format(x_value, '06b')] for x_value in x_values]
```

The fitness evaluation for this task should be performed as follows:
The integer value (phenotype) that is represented via the 6-bit string (genotype) takes a value from 0 to 63. In order to get the fitness value, the following steps are performed:

$$y = \begin{cases} x - 64, & \text{if } x \geq 32 \\ x, & \text{otherwise} \end{cases} \tag{1}$$

$$\begin{aligned} \text{fitness} = & -0.00008333y^6 - 0.001y^5 + 0.09375y^4 \\ & + 1.16667y^3 - 19.53125y^2 - 234.375y + 8350 \end{aligned} \tag{2}$$

In short, each x-value (in phenotype space; represented via a 6-bit string in genotype space) is first adjusted for values of 32 or larger, and then this value y is plugged into a simple polynomial function to get the fitness value. (Note: This two-step procedure is used as it helps to get a very interesting fitness landscape that we can work on).

For completing this first task, the following functions (and loop) are required:

1. **Initialization Function**: (Inputs: Population Size; Output: Individuals). This separate function *initialization_population* will create the individuals for the population, each having six binary genes. It should return the individuals, for example as a list of lists e.g., [[0, 1, 1, 0, 1,0], [0, 1, 1, 1, 1,1], [1, 0, 1, 0, 1,1]].
   Subtask: Use this function to generate 4 individuals and state those specific individuals generated (and the type of output e.g., a list of lists).

2. **Fitness Function**: (Inputs: Individuals; Output: Fitness values). This separate function *fitness* takes as input one or multiple individuals (genotype), uses the *geno_to_pheno* function to convert them to the integer value (phenotype) and calculates the fitness value for all of them (as specified above).
   Subtask: Check that for *individual = [[0, 1, 0, 1, 0, 1]]* the fitness value is approximately *12620.97*. Also, plot the fitness landscape (x-axis: integer values (phenotype); y-axis: fitness) based on all possible integer values (phenotype). State also what x-values correspond to the minimum and to the maximum (fitness).

3. **Parent Selection Function**: (Inputs: Individuals, Fitness values; Output: Parents (in random order, same number as individuals)). This separate function *parent_selection* should perform fitness-proportional parent selection (as discussed in class) so that more fit parents have a higher probability to be selected. One function that may be of help here is *numpy.random.choice* from the *numpy* module.
   Subtask: You saw from the previous subtask the fitness landscape. Test the parent selection by providing as the set of four individuals: the one leading to minimum fitness, to maximum fitness and two more individuals of moderate fitness (middle of fitness range). State which specific 4 individuals are in the set you tested, their fitness values and the parents the function provided. Hint: The highest fitness individual should (usually) be selected more often than the minimum fitness one (select a suitable run to show this).

4. **Recombination (Crossover) Function**: (Inputs: Parents, Crossover Probability P_C; Outputs: Offspring (pre-Mutation)). This separate function *one_point_crossover* performs 1-point crossover and generates for parents 1 & 2, parents 3 & 4 etc. offspring (and e.g., provides all offspring as a list of lists). Keep in mind that you can use a random uniform number $\in [0, 1]$ and compare it against the crossover probability, if it does not exceed it, crossover is performed (with a random cutoff point, using *randint* function in *random* module), otherwise, the two parents are simply copied to be the offspring.
   Subtask: Use two parents as input and try out the function. State (as genotypes) the two parents, the offspring, and where the random cutoff point apparently was (you can deduce it by comparing the parents with the child, if you select suitable parents).

5. **Mutation Function**: (Inputs: Individuals, Mutation Probability P_M; Outputs: Offspring (mutated)). This separate function *mutation* performs bitflip mutation for each bit / gene and each individual with mutation probability P_M. Similarly to crossover, keep in mind that you can use a random uniform number $\in [0, 1]$ and compare it against the mutation probability, if it does not exceed it, the value for the gene is flipped (e.g., from 0 to 1 or vice versa).
   Subtask: Use one individual as input and perform mutation for that individual - once with probability of 0.2 and once with a probability of 0.9. State (as a genotype) the original individual and the two (mutated) individuals. Comment on whether the number of mutations that occurred

corresponds to your expectation and why.

6. **Survivor Selection Function**: (Inputs: Individuals, Offspring, Fitness Individuals, Fitness Offspring; Outputs: (New) population). This separate function *survivor_selection* performs deterministic survivor selection (no randomness / stochasticity here) by taking from among the population individuals and the offspring the survivors based on the highest fitness values (population size should remain the same!).
   Subtask: Provide to the function one current population individual and one offspring, together with their respective fitness values. State both individuals (genotypes), their fitness values, and the one survivor returned, ensuring that the fitter individual was selected to survive.

7. **Loop**: Now create the complete version of the simple genetic algorithm (SGA), using the functions previously coded. Specifically, initialize the population and then loop (for a number of iterations to be specified) over those steps that should be repeated in the SGA.
   Subtask: Make sure to display the average and best fitness in each iteration. Is the average and best fitness non-decreasing? Why / why not? Is this according to your expectation (knowing the specifics of the SGA implementation). Why / why not?

8. Use the SGA you have just programmed and try out different combinations for the *number of iterations* and the *population size*. Show your results and comment on how the number of iterations and population size tend to affect the best and average fitness achieved after the SGA terminated. Comment on your findings and explain them.

4 Task 2: Knapsack for "data_1001" data set

This task focuses on the *"data_1001"* data set, representing a knapsack problem with 1001 items. The problem concerns which items to select to maximize the fitness (overall value of selected items) using the SGA programmed in the previous task.
(Optionally, the *"data_31"* data set is available for simple testing of your GA to verify it works as expected)

The following tasks should be considered:

1. Load the data set *data_1001.csv*. The first column contains the values for each item - save them as a list *item_values* - and the second column contains the weights of each item - save them as a list *item_weights*. State the size of the data set and analyze the values and the weights that the items take and any possible relationship between the values and the weights. Comment on your findings.

2. Write a **NEW initialization function** that creates a binary list that is suitable to represent this problem. As before: (Inputs: Population Size, Output: Individuals).

3. Since the aim of knapsack is to maximize the value of items selected (0 - not selected, 1 - selected), write a **NEW fitness function** that sums the value of all items selected.

4. Run the SGA with the new initialization and new fitness function, with population size 30, number of iterations 1000, $P_C = 0.9$ and $P_M = 0.1$. State the average fitness of the final population and the best fitness in the final population. (Note: Since the SGA is heuristic, different groups may end up with different best solutions and best fitness values).

5. The best solution found is harder to interpret than before (as it cannot just be converted into a single integer). Thus, describe the solution found based on the fitness, number of items selected, and the min, mean, max of the values, of the weights and of the ratio of value-to-weight of the selected items. Compare this with the same information on the items that were not selected. Comment on your findings and whether the results are as expected. Why / why not?

5 Task 3: Knapsack "data_1001" with Constraint

The previous problem should next be adjusted to include a **constraint** on the **exact number of items** that need to be selected for the knapsack (e.g., each individual has exactly 10 items, not more, not less). Assume that no population member or offspring may violate the constraint on the exact number of items. The following tasks should be considered:

1. Discuss which functions of the SGA (i.e., initialization, fitness, parent selection, recombination, mutation, survivor selection) need to be updated so that the constraint on the exact number of items is not violated by any population member or offspring. Comment for each whether it needs to be updated, why the original function version will or may lead to a violation of the constraint and how it could be updated **(Note: Full-point solutions must satisfy the constraint immediately after recombination and immediately after mutation, without first creating invalid offspring. Solutions that temporarily create invalid offspring e.g., by using bitflip (as in task 2), and subsequently repair them (e.g., at random or otherwise) are acceptable but will <u>not</u> receive full points.)**.

2. According to the answer above, create the updated functions for the SGA (Note: briefly check for each updated function with a simple example that it performs as intended). Run the algorithm for at least the following exact number of items $[1001, 802, 400, 100, 10]$ and record (1) the fitness, (2) number of items, and (3) average value per selected item (i.e., fitness / number of items). Comment on how the number of items in the constraint affects the fitness and the average value per selected item.

3. Assume now that instead of a constraint for the exact number of items, there would be a **constraint for the upper bound of the number of items** OR a **constraint for the upper bound of the summed weight** of selected items in the knapsack (e.g., not more than 100 items OR not more than 100'000 summed weight of selected items). For both cases, discuss now (1) which functions would have to be updated and (2) how each function specifically could be updated so that the constraint is at no point violated.

4. **BONUS (10 points)**: Implement a new version of the SGA for one of the two problems (Task 3, subtask 3). Discuss with reference to the best result from the SGA with the exact number of items constraint (Task 3, subtask 1 & subtask 2), how the solution found is different (e.g., in terms of the summed weight or the number of items found) and briefly explain why the result may be better or worse.

6 Report

Write a report. It does **not** have to be structured like a research paper. Rather structure it according to the tasks and describe the process, data, results, answers to the questions and discuss your findings. Make sure for each task to also describe the functions you created and the logic behind them.